Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»
Факультет компьютерных систем и сетей
Кафедра информатики

Дисциплина «Операционные системы и системное программирование»

ОТЧЕТ

к лабораторной работе №5

на тему:

«Децентрализованный TCP-чат»

Вариант 6

Выполнил: студент группы _____
Дмитрий Василенко
Проверил: _____

Минск, 2026

ЦЕЛЬ РАБОТЫ

Изучить принципы организации сетевого взаимодействия процессов и получить практические навыки разработки клиентских и серверных приложений с использованием программного интерфейса сокетов.

В ходе работы необходимо:

- изучить модели OSI и TCP/IP, назначение протоколов TCP и UDP;
- освоить программный интерфейс сокетов и его аналоги в Linux;
- реализовать взаимодействие независимых процессов по сети;
- разработать прикладной протокол с собственным кадрированием сообщений;
- реализовать равноправные узлы, пересылку сообщений, TTL и дедупликацию;
- обработать сетевые ошибки, разрывы соединений и тайм-ауты;
- провести эксперименты и оценить задержку и пропускную способность;
- оформить исходный код, данные, графики и историю Git.

1. ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Вариант 6 — децентрализованный TCP-чат.

Требования варианта:

- протокол TCP;
- не менее трёх равноправных узлов, без выделенного центрального сервера;
- каждый узел одновременно принимает входящие TCP-соединения, устанавливает исходящие соединения, отправляет, принимает и пересылает сообщения;
- сообщение содержит `messageId`, идентификатор отправителя, адресата, текст и TTL — максимальное число оставшихся переходов;
- допускается передача через промежуточные узлы;
- необходимо предотвращать бесконечную пересылку с помощью TTL и учёта ранее обработанных `messageId`;
- эксперименты: цепочки из 3 и 4 узлов;
- проверить доставку через промежуточный узел, обработку неизвестного адресата и прекращение пересылки по TTL.

Индивидуальные параметры:

- `STUDENT_ID = 028`;
- `SEED = 1000 + 28 * 97 = 3716`;
- `BASE_PORT = 20000 + 28 = 20028`;
- основной язык — C++17, среда — Ubuntu/Linux, сокеты POSIX.

2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Модель OSI делит сетевые функции на семь уровней. В данной работе используются прикладной уровень (собственный протокол чата), транспортный уровень (TCP) и сетевой уровень (IP). Стек TCP/IP объединяет эти уровни практически: приложение работает с TCP-сокетом, а IP обеспечивает адресацию и маршрутизацию пакетов.

TCP предоставляет упорядоченный поток байтов, а не последовательность самостоятельных сообщений. Один вызов `send()` не гарантирует передачу всех байтов, а один вызов `recv()` не гарантирует получение целого сообщения приложения. Поэтому в работе реализованы:

- 4-байтовое поле длины в сетевом порядке байтов;
- функции `send_all()` и `recv_exact()`, обрабатывающие частичные операции;
- проверка допустимого размера кадра;
- собственный формат полей `key=value` в UTF-8.

Сокет — это программная абстракция, объединяющая IP-адрес и номер порта. Для TCP-сервера последовательность вызовов обычно имеет вид `socket -> bind -> listen -> accept`, для клиента — `socket -> connect`. После установления соединения обе стороны используют `send`/`recv`, а завершают работу через `shutdown` и `close`.

В одноранговой (peer-to-peer) архитектуре нет центрального сервера: каждый узел одновременно является и «сервером» для входящих соединений, и «клиентом» для исходящих. Пересылка сообщений через промежуточные узлы требует защиты от петель. В работе используются два механизма: уменьшение TTL и множество уже обработанных `messageId`.

3. АРХИТЕКТУРА СИСТЕМЫ

Система состоит из одинаковых процессов `peer_chat`. Каждый процесс имеет уникальный идентификатор (`--id`) и прослушивает собственный порт (`--port`). Список соседей задаётся повторяющимся параметром `--peer host:port`. Центральный сервер отсутствует.

Основные компоненты узла:

- **поток прослушивания** — принимает входящие TCP-соединения через `select`/`accept`;
- **потоки подключения** — для каждого соседа из `--peer` пытаются установить соединение с тайм-аутом и повторяют попытки при отказе;
- **поток чтения соединения** — для каждого принятого или установленного соединения читает кадры, разбирает сообщения и выполняет пересылку;
- **основной поток** — реализует интерактивные команды или автоматическую отправку сообщений;
- **общее состояние** — вектор соединений, множество обработанных `msgid`, счётчики и журнал; доступ защищён мьютексами, счётчики — атомарные.

Схема цепочек приведена на рисунках 1 и 2. Направленные рёбра означают исходящие TCP-соединения, созданные параметром `--peer`.

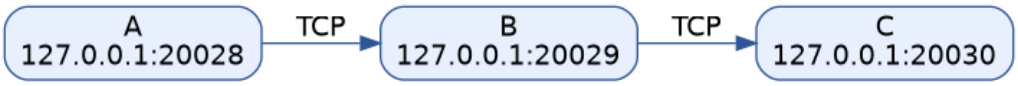


Рисунок 1. Топология цепочки из трёх узлов



Рисунок 2. Топология цепочки из четырёх узлов

4. ОПИСАНИЕ КЛИЕНТСКОЙ И СЕРВЕРНОЙ ЧАСТЕЙ

В варианте 6 клиентская и серверная роли объединены в одном процессе. «Серверная» часть — это прослушивание порта и приём входящих соединений. «Клиентская» часть — это установление исходящих соединений с соседями и отправка сообщений.

После установления TCP-соединения обе стороны отправляют сообщение `HELLO` со своим идентификатором. Получив `HELLO`, узел запоминает идентификатор соединения. Это позволяет пересылать сообщения только по уже инициализированным соединениям и не отправлять их в соединение, из которого пришло сообщение.

Каждое соединение имеет собственный мьютекс записи. Это нужно потому, что одно и то же сообщение может пересылаться несколькими потоками, а `send()` не является потокобезопасным для одновременных вызовов. Чтение выполняется отдельным потоком на соединение, поэтому медленный или закрывшийся сосед не блокирует остальные соединения.

Основные фрагменты реализации:

```
int PeerNode::open_listener(std::string& error) {
    return create_listener(config_.host, config_.port, error);
}

void PeerNode::listener_loop() {
    while (!stopping()) {
        fd_set read_set;
        FD_ZERO(&read_set);
        FD_SET(listener_fd_, &read_set);
        timeval timeout{0, 200000};
        const int selected = ::select(listener_fd_ + 1, &read_set,
                                      nullptr, nullptr, &timeout);
        if (selected <= 0) continue;
        sockaddr_storage address{};
        socklen_t length = sizeof(address);
        const int client_fd = ::accept(listener_fd_, ...);
        // ...
    }
}
```

При завершении работы узел выставляет флаг остановки, закрывает слушающий сокет, выполняет `shutdown` для активных соединений, дожидается завершения потоков, закрывает оставшиеся дескрипторы и записывает итоговую сводку.

5. ОБОСНОВАНИЕ ВЫБОРА TCP

Для варианта выбран TCP, поскольку:

- требуется доставка сообщений через цепочку узлов без потерь;
- TCP обеспечивает упорядоченность и повторную передачу на транспортном уровне, поэтому прикладной протокол не должен реализовывать собственные подтверждения и повторные отправки;
- TCP удобен для постоянных соединений между равноправными узлами;
- для проведения экспериментов важно отделить сетевые потери от логики приложения.

UDP не используется, так как он не гарантирует доставку и порядок, а вариант не требует дейтаграммного обмена. При этом TCP не сохраняет границы сообщений, поэтому в работе реализовано собственное кадрирование `[Length][Payload]` и корректная обработка частичного приёма.

6. СЕТЕВОЙ ПРОТОКОЛ

Протокол прикладного уровня имеет версию 1. Каждое сообщение передаётся одним кадром:

```
+------------------+-----------------------------+
| Length: 4 байта  | Payload: Length байт        |
| network order    | UTF-8                       |
+------------------+-----------------------------+
```

Поле `Length` — беззнаковое 32-битное целое в сетевом порядке байтов (`htonl`/`ntohl`). Допустимый размер — от 1 до 65536 байт. Payload состоит из строк `key=value`, разделённых `\n`; значения экранируются.

Таблица 1 — Типы сообщений

Тип	Отправитель	Получатель	Содержимое	Ожидаемая реакция
`HELLO`	любой узел	любой узел	`id`	сохранить идентификатор соединения
`CHAT`	любой узел	любой узел	`msgid`, `dest`, `ttl`, `text`, `sent_us`	доставить, переслать, отбросить
`ERROR`	любой узел	любой узел	`code`, `text`	записать в журнал

Таблица 2 — Поля сообщения `CHAT`

Поле	Тип	Ограничение	Назначение
`msgid`	строка	не пустая	уникальный идентификатор сообщения
`dest`	строка	не пустая	идентификатор адресата
`ttl`	целое	0..64	число оставшихся переходов
`text`	строка	до 4000 байт	текст сообщения
`sent_us`	целое	>= 0	метка отправки для измерения задержки

Правила обработки `CHAT`:

1. Проверить наличие обязательных полей и допустимость значений.
2. Если `msgid` уже встречался — увеличить `dropped_dup` и не обрабатывать сообщение повторно.
3. Если `dest` совпадает с идентификатором узла — доставить сообщение, увеличить `delivered`.
4. Иначе, если `ttl > 0`, уменьшить TTL на 1 и переслать сообщение всем активным соединениям, кроме входящего; увеличить `forwarded`.
5. Иначе, если `ttl == 0`, отбросить сообщение и увеличить `dropped_ttl`.
6. Если пересылать некуда, увеличить `dropped_no_route`.

Отправитель также запоминает `msgid` своего сообщения, чтобы не принять его обратно и не начать повторную пересылку.

Таблица 3 — Коды ошибок протокола

Код	Причина
`BAD_MESSAGE`	отсутствует обязательное поле, неверный TTL, неизвестный тип или слишком длинный текст
`INVALID_FRAME`	неверная длина кадра; соединение закрывается

7. СТРУКТУРА ПЕРЕДАВАЕМЫХ СООБЩЕНИЙ

Пример сообщения от узла A узлу C через узел B:

```
type=CHAT
ver=1
id=A
msgid=A-1794235576839-0
dest=C
ttl=5
text=payload #1
sent_us=1794235576839123
```

После пересылки узлом B поле `ttl` уменьшается до 4, остальные поля сохраняются. Поле `sent_us` позволяет получателю вычислить полное время от вызова отправки до доставки. В распределённой системе для такого измерения потребовалась бы синхронизация часов; в данной работе все узлы запускаются на одной машине.

8. ДИАГРАММА ВЗАИМОДЕЙСТВИЯ

На рисунке 3 показана последовательность для цепочки A–B–C: установление соединений и обмен `HELLO`, успешная доставка, неизвестный адресат, исчерпание TTL и завершение. На рисунке 4 — аналогичная цепочка из четырёх узлов.

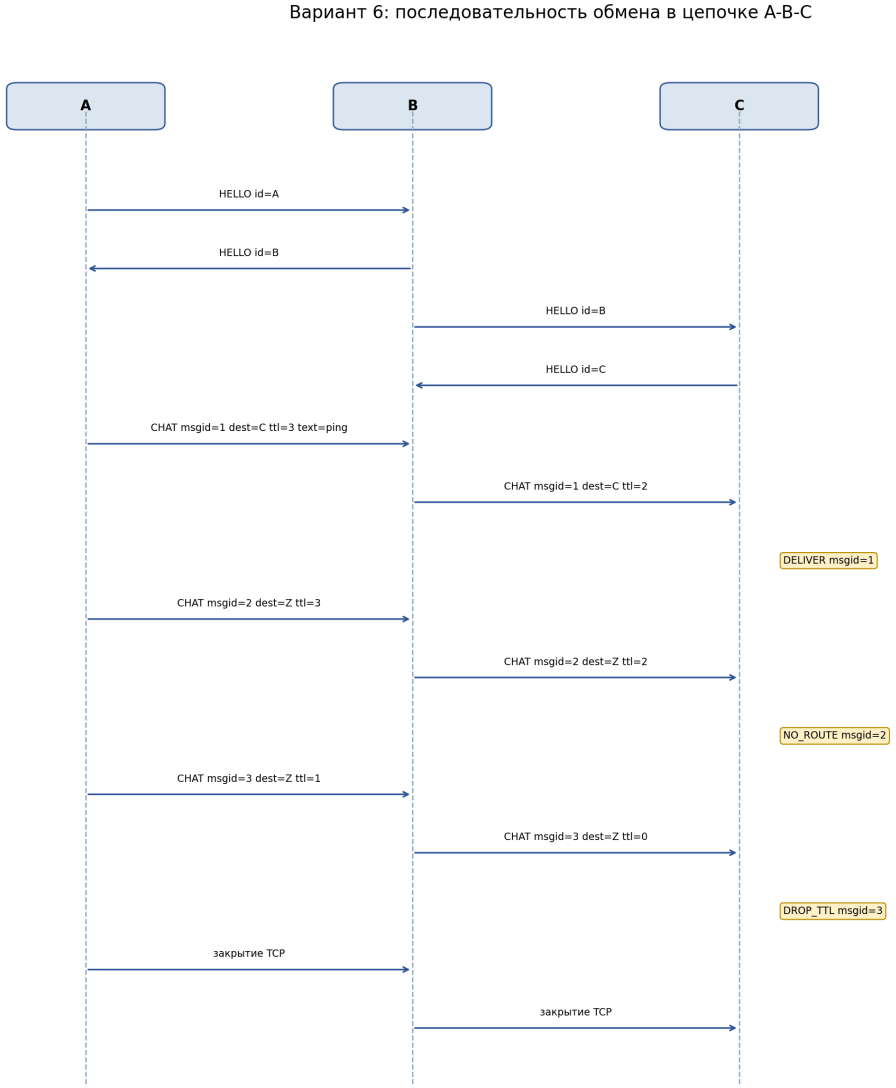


Рисунок 3. Диаграмма последовательности для трёх узлов

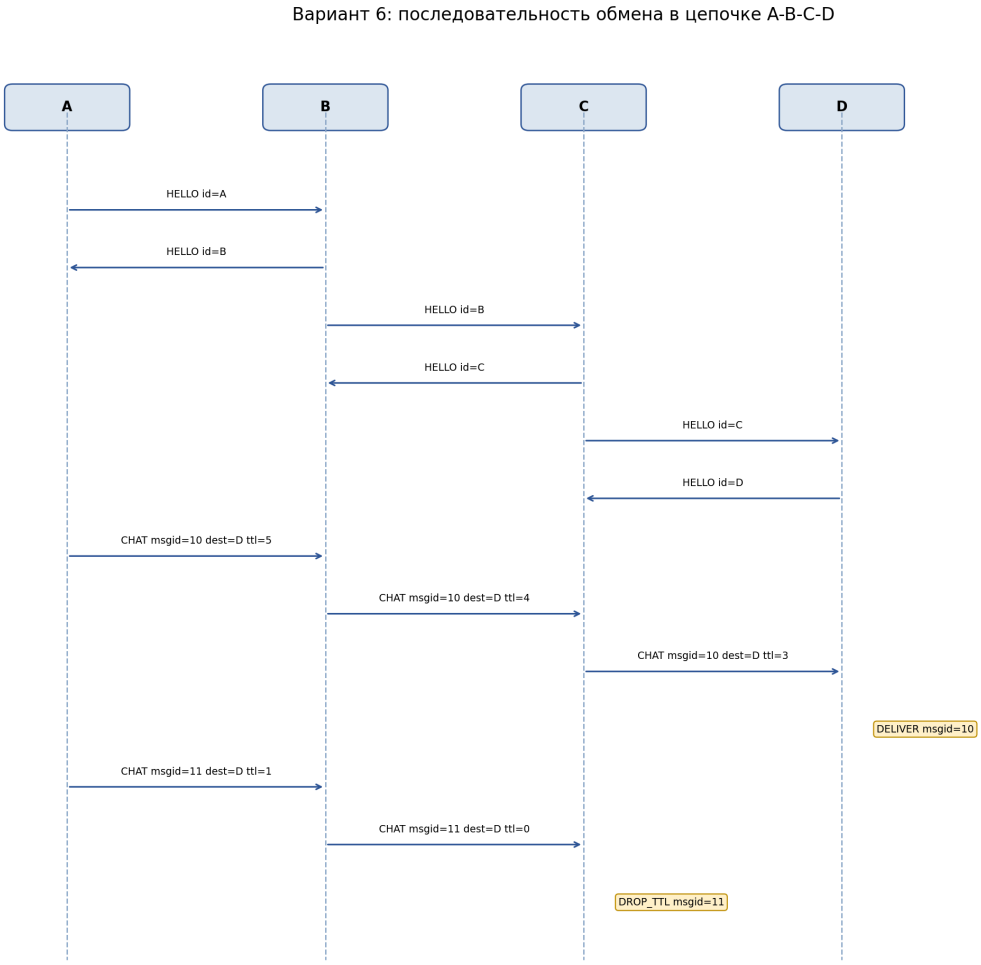


Рисунок 4. Диаграмма последовательности для четырёх узлов

Исходные описания диаграмм находятся в файлах `diagrams/sequence_chain3.puml` и `diagrams/sequence_chain4.puml` (PlantUML), а также в скрипте `scripts/render_diagrams.py`, которым построены изображения.

9. ИСПОЛЬЗОВАННЫЕ ФУНКЦИИ И ИХ АНАЛОГИ

Лабораторное задание ориентировано на Windows Sockets API. Работа выполнена на Ubuntu/Linux, поэтому использованы POSIX-аналоги. Соответствие приведено в таблице 4.

Таблица 4 — Соответствие функций Winsock и POSIX

Windows	**Linux/POSIX**	**Назначение**
`WSAStartup`	—	в Linux инициализация не требуется
`WSACleanup`	—	ресурсы освобождаются ядром
`socket`	`socket`	создание сокета
`bind`	`bind`	привязка к адресу и порту
`listen`	`listen`	перевод в режим ожидания
`accept`	`accept`	приём входящего соединения
`connect`	`connect`	установление соединения
`send`	`send`	передача данных
`recv`	`recv`	приём данных
`sendto/recvfrom`	—	в TCP-варианте не используются
`select`	`select`	ожидание готовности сокетов
`shutdown`	`shutdown`	прекращение передачи
`closesocket`	`close`	закрытие сокета
`WSAGetLastError`	`errno + strerror`	код и текст ошибки
`getaddrinfo`	`getaddrinfo`	разрешение адреса
`getnameinfo`	`getnameinfo`	адрес в текст
`setsockopt`	`setsockopt`	тайм-ауты, `SO_REUSEADDR`
`ioctlsocket`	`fcntl`	неблокирующий режим
`QueryPerformanceCounter`	`std::chrono`	измерение времени

10. ОБРАБОТКА ОШИБОК И ТАЙМ-АУТОВ

Ошибки и тайм-ауты обрабатываются на нескольких уровнях. При создании сокета, связывании и прослушивании выводится событие `LISTEN_ERROR` с функцией, кодом `errno` и текстом `strerror(errno)`. При подключении используется неблокирующий `connect` с `select` и тайм-аутом; при отказе узел пишет `CONNECT_FAIL` и повторяет попытку через 300 мс.

Таблица 5 — Обработка сетевых ситуаций

Ситуация	**Поведение программы**
занятый порт	`LISTEN_ERROR`, завершение с кодом 1
недоступный сервер	`CONNECT_FAIL`, повторные попытки до остановки
закрытие соединения	`PEER_CLOSED`, `DISCONNECTED`, удаление из списка
тайм-аут подключения	ошибка `connect timeout`, новая попытка
тайм-аут приёма	`recv` с `SO_RCVTIMEO`; ожидание продолжается
ошибка отправки	`SEND_ERROR`, `shutdown` соединения
некорректное сообщение	`PROTOCOL_ERROR`, ответ `ERROR`, соединение сохраняется
неверная длина кадра	`RECV_ERROR`, соединение закрывается
неизвестный тип	ошибка разбора, ответ `ERROR`
дубликат	`DUP`, повторная обработка не выполняется
TTL = 0	`DROP_TTL`
нет маршрута	`NO_ROUTE`

Основа обработки частичного приёма:

```
IoResult recv_exact(int fd, char* buffer, std::size_t length,
                    std::string& error) {
    std::size_t total = 0;
    while (total < length) {
        const ssize_t received =
            ::recv(fd, buffer + total, length - total, 0);
        if (received > 0) {
            total += static_cast<std::size_t>(received);
            continue;
        }
        if (received == 0) return IoResult::Closed;
        if (errno == EINTR) continue;
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            return total == 0 ? IoResult::Timeout : IoResult::Error;
        }
        error = std::string("recv: ") + std::strerror(errno);
        return IoResult::Error;
    }
    return IoResult::Ok;
}
```

11. МЕТОДИКА ТЕСТИРОВАНИЯ

Сборка и автоматические проверки:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure
```

Таблица 6 — Автоматические проверки

Проверка	**Что проверяется**
доставка через промежуточный узел	A -> B -> C, событие `DELIVER` у C и `FORWARD` у B
неизвестный адресат	сообщение для Z не доставляется, узел C пишет `NO_ROUTE`
исчерпание TTL	при TTL = 1 узел C пишет `DROP_TTL`
дубликаты	два одинаковых `msgid`; второе сообщение отбрасывается
ошибка протокола и частичный приём	некорректное сообщение даёт `ERROR`, побайтовая передача собирается
недоступный сервер	узел пишет `CONNECT_FAIL` и корректно завершается

Эксперименты проводятся на цепочках из 3 и 4 узлов. Для каждой конфигурации выполняется три запуска по 20 сообщений с интервалом 50 мс, TTL = 5, фиксированным `SEED = 3716`. Измеряется интервал от вызова отправки на узле A до записи `DELIVER` у конечного узла. Дополнительно собираются счётчики доставленных и пересланных сообщений, ошибок и объёма переданных байтов.

12. ЭКСПЕРИМЕНТАЛЬНЫЕ РЕЗУЛЬТАТЫ

Таблица 7 — Средняя задержка доставки, мс

Конфигурация	Запуск 1	Запуск 2	Запуск 3	Среднее
Цепочка A–B–C	0.439	0.548	0.536	0.508
Цепочка A–B–C–D	0.742	0.691	0.744	0.726

Таблица 8 — Сводные показатели экспериментов

Конфигурация	Отпр.	Дост.	Перес.	Ошиб.	Байт	Сообщ./с
A–B–C	20	20	20	0	2107	17.32
A–B–C–D	20	20	40	0	2107	17.31

Отпр. — отправлено, Дост. — доставлено, Перес. — число пересылок, Ошиб. — число ошибок.

Во всех шести запусках доставлены все 20 сообщений. Ошибок протокола, потерь и дубликатов не было. Число пересылок в цепочке из четырёх узлов вдвое больше: сообщение проходит через два промежуточных узла, тогда как в цепочке из трёх — через один. Объём переданных байтов у отправителя и получателя совпадает, что подтверждает корректность кадрирования.

13. ГРАФИКИ И АНАЛИЗ

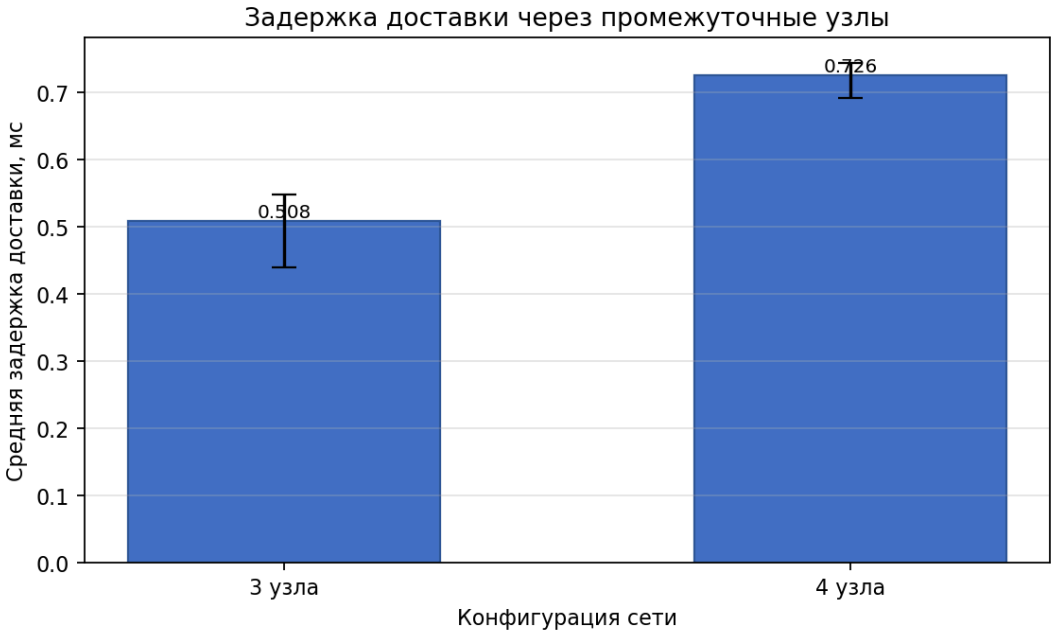


Рисунок 5. Средняя задержка доставки в цепочках из 3 и 4 узлов



Рисунок 6. Пропускная способность в цепочках из 3 и 4 узлов

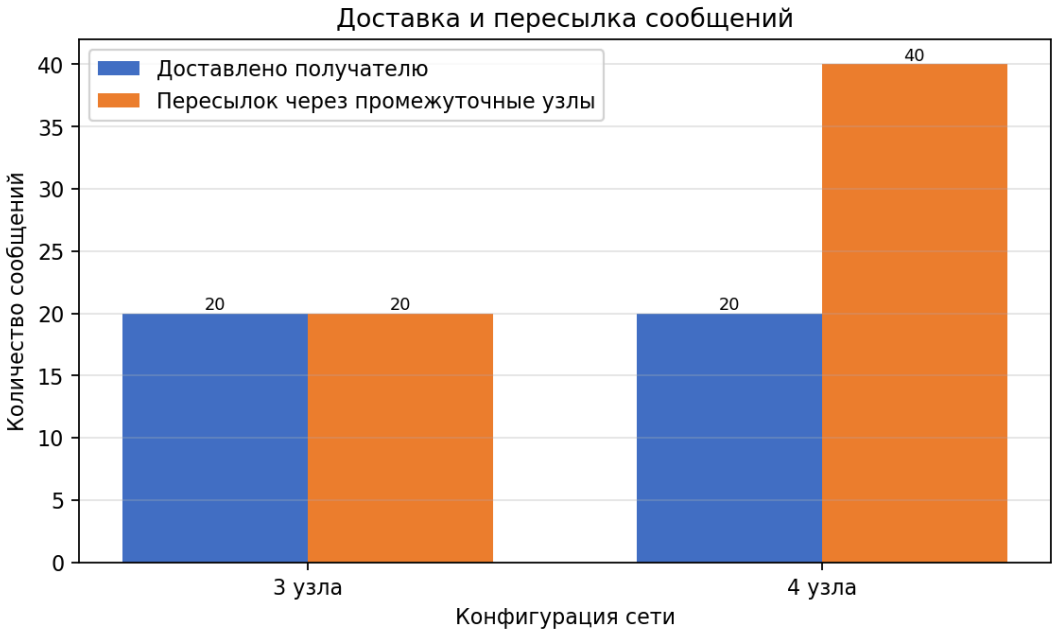


Рисунок 7. Доставленные и пересланные сообщения

Средняя задержка растёт с 0.508 мс для трёх узлов до 0.726 мс для четырёх узлов, то есть примерно на 43 %. Это объясняется дополнительным переходом A -> B -> C -> D и дополнительной обработкой сообщения на промежуточном узле. Абсолютные значения малы, потому что все процессы запущены на одной машине и обмен идёт через loopback-интерфейс.

Пропускная способность практически не меняется (17.32 и 17.31 сообщений/с), поскольку она ограничена заданным интервалом отправки 50 мс, а не сетью. Теоретический предел при таком интервале — около 20 сообщений/с; измеренное значение немного ниже из-за времени установления соединений и первого сообщения.

Число пересылок показывает overhead маршрутизации: 20 пересылок для цепочки из трёх узлов и 40 для цепочки из четырёх. Число доставленных сообщений одинаково и равно 20. Ошибок и потерь нет, что подтверждает корректность обработки TTL, дедупликации и частичного приёма.

14. ВЫВОДЫ

В ходе лабораторной работы разработан децентрализованный TCP-чат из равноправных узлов. Реализованы прослушивание TCP-порта, установление исходящих соединений, обмен `HELLO`, отправка и приём сообщений, пересылка через промежуточные узлы, уменьшение TTL, учёт `messageId` и обработка сетевых ошибок.

Применение TCP позволило сосредоточиться на прикладной логике: транспорт обеспечивает упорядоченность и доставку байтов, а собственный протокол задаёт границы сообщений, версию, идентификаторы и правила маршрутизации. Реализованы корректная обработка частичного `send`/`recv`, тайм-ауты подключения и отправки, ответы `ERROR` на некорректные сообщения и журналирование основных событий.

Автоматические проверки подтвердили доставку через промежуточный узел, обработку неизвестного адресата, прекращение пересылки по TTL, подавление дубликатов и устойчивость к ошибкам протокола. Эксперименты на цепочках из 3 и 4 узлов показали рост задержки примерно на 43 % при добавлении одного промежуточного узла и двукратный рост числа пересылок при отсутствии потерь. Цель работы достигнута, результаты воспроизводимы, исходные данные и скрипты построения графиков сохранены в репозитории.

15. ИСПОЛЬЗОВАНИЕ МОДЕЛЕЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

Инструмент: opencode (CLI-агент).

Модель: в интерфейсе указан идентификатор `opencode/deepseek-v4.1-flash-free`. Отдельная версия модели в интерфейсе не отображается, поэтому более точная версия не заявляется.

Цель использования: проектирование структуры лабораторной работы, генерация исходного кода на C++17, тестов, скриптов экспериментов и визуализации, подготовка черновика отчёта и раздела об использовании ИИ.

Оригинальные промпты: полный текст исходного запроса пользователя приведён в файле `report/ai_prompts.md` (Приложение Д). Запрос содержал описание лабораторной работы, индивидуальное задание варианта 6, требования к отчёту, экспериментам, Git и разделу об использовании ИИ, а также указание выполнить работу на Ubuntu с использованием C++ и POSIX-аналогов Windows Sockets API.

Полученный результат: структура проекта, исходный код узла и общего слоя, автоматические проверки, скрипты запуска экспериментов и построения графиков, диаграммы, текст отчёта и README.

Место использования: каталоги `src/`, `tests/`, `scripts/`, `diagrams/`, `report/`, файлы `README.md`, `docs/protocol.md`, `docs/plan.md`.

Самостоятельная проверка: проект собран компилятором GCC 13.3.0, выполнены `ctest` (6 проверок), ручные запуски цепочек из 3 и 4 узлов, проверены журналы `DELIVER`, `FORWARD`, `DROP_TTL`, `NO_ROUTE`, `DUP`, `PROTOCOL_ERROR`; эксперименты повторены три раза для каждой конфигурации, проверено соответствие CSV и графиков.

Самостоятельные изменения: в процессе проверки исправлено завершение интерактивного режима при отсутствии stdin (узел теперь корректно доживает до `--run-ms`), исправлен тестовый сценарий с повторным запуском узла A, добавлено поле `sent_us` для измерения задержки с микросекундной точностью, синхронизированы имена событий и счётчиков в журнале и скриптах.

16. СВЕДЕНИЯ О РЕПОЗИТОРИИ GIT

Таблица 9 — Сведения о репозитории

Параметр	**Значение**
Ссылка на репозиторий	https://github.com/Dima-Vasilenko/osisp
Каталог лабораторной работы	`lab5/`
Ветка	`main`
Этап 1. Проектирование	`7cd5a34` — структура, README, протокол, план
Этап 2. Базовый слой	`bee2b80` — кадрирование, протокол, журнал, сокеты
Этап 2–3. Сетевое ядро	`51d0445` — узел, соединения, пересылка, TTL
Этап 3. Тестирование	`bce3a20` — автоматические проверки
Этап 4. Эксперименты	`b9a3d7f` — сырые данные, `3a56cc5` — графики и диаграммы
Итоговый коммит с кодом и данными	`3a56cc5`

Сборка и запуск:

```
cmake -S lab5 -B lab5/build -DCMAKE_BUILD_TYPE=Release
cmake --build lab5/build -j
ctest --test-dir lab5/build --output-on-failure
python3 lab5/scripts/run_experiments.py
python3 lab5/scripts/plot_results.py
```

17. ПРИЛОЖЕНИЯ

Приложение A. Ключевые фрагменты кода

Кадрирование и частичная отправка:

```
bool send_all(int fd, const char* data, std::size_t length,
              std::string& error) {
    std::size_t total = 0;
    while (total < length) {
        const ssize_t sent =
            ::send(fd, data + total, length - total, MSG_NOSIGNAL);
        if (sent > 0) { total += static_cast<std::size_t>(sent); continue; }
        if (sent < 0 && errno == EINTR) continue;
        error = std::string("send: ") + std::strerror(errno);
        return false;
    }
    return true;
}
```

Проверка дубликатов и пересылка:

```
{
    std::lock_guard<std::mutex> lock(seen_mutex_);
    if (!seen_messages_.insert(message.msgid).second) {
        stats_.dropped_dup.fetch_add(1);
        return;
    }
}
if (message.dest == config_.id) {
    stats_.delivered.fetch_add(1);
    // вывод сообщения и запись DELIVER
    return;
}
if (message.ttl > 0) {
    forward_chat(connection, message, message.ttl - 1);
} else {
    stats_.dropped_ttl.fetch_add(1);
}
```

Тайм-аут подключения:

```
const int selected = ::select(fd + 1, nullptr, &writable, nullptr, &timeout);
if (selected == 0) {
    error = "connect timeout";
    ::close(fd);
    continue;
}
int socket_error = 0;
socklen_t length = sizeof(socket_error);
::getsockopt(fd, SOL_SOCKET, SO_ERROR, &socket_error, &length);
if (socket_error != 0) {
    error = std::string("connect: ") + std::strerror(socket_error);
}
```

Полный исходный код находится в каталогах `src/`, `tests/` и `scripts/` репозитория.

Приложение Б. Исходные экспериментальные данные

Таблица Б.1 — Счётчики доставки и пересылки

Конфиг.	Запуск	Узлов	Отправлено	Доставлено	Пересылок	Отброшено	Ошибок
chain3	1	3	20	20	20	0 / 0 / 0	0
chain3	2	3	20	20	20	0 / 0 / 0	0
chain3	3	3	20	20	20	0 / 0 / 0	0
chain4	1	4	20	20	40	0 / 0 / 0	0
chain4	2	4	20	20	40	0 / 0 / 0	0
chain4	3	4	20	20	40	0 / 0 / 0	0

В столбце «Отброшено» через дробь указаны значения для исчерпания TTL, дубликатов и отсутствия маршрута соответственно.

Таблица Б.2 — Задержки и объём переданных данных

Конфиг.	Запуск	Сред., мс	Мин., мс	Макс., мс	Длит., мс	msg/s	Байт
chain3	1	0.439	0.331	0.608	1152.000	17.361	2107
chain3	2	0.548	0.376	0.873	1156.000	17.301	2107
chain3	3	0.536	0.366	0.808	1156.000	17.301	2107
chain4	1	0.742	0.612	0.890	1155.000	17.316	2107
chain4	2	0.691	0.526	0.855	1157.000	17.286	2107
chain4	3	0.744	0.574	1.384	1154.000	17.331	2107

Полные файлы `data/raw/experiment_raw.csv` и `data/raw/latency_raw.csv` содержат те же поля в машиночитаемом виде, а также задержки всех 120 доставленных сообщений (по 20 сообщений в каждом из шести запусков).

Приложение В. Редактируемые диаграммы

Исходное описание PlantUML для рисунка 3:

```
@startuml
title Вариант 6: доставка сообщения в цепочке из трёх узлов
skinparam monochrome true
skinparam sequenceMessageAlign center

participant "Узел A\nid=A" as A
participant "Узел B\nid=B" as B
participant "Узел C\nid=C" as C

note over A, C: Запуск узлов и установление TCP-соединений
A -> B: HELLO id=A
B -> A: HELLO id=B
B -> C: HELLO id=B
C -> B: HELLO id=C

note over A, C: Сообщение, адресованное узлу C
A -> B: CHAT msgid=1 dest=C ttl=3 text=ping
note right of B: dest != B, TTL > 0\nmsgid ещё не встречался
B -> C: CHAT msgid=1 dest=C ttl=2
note right of C: dest = C\nDELIVER, запись в журнал

note over A, C: Неизвестный адресат
A -> B: CHAT msgid=2 dest=Z ttl=3
```

```
B -> C: CHAT msgid=2 dest=Z ttl=2
note right of C: Маршрута нет\nNO_ROUTE

note over A, C: Исчерпание TTL
A -> B: CHAT msgid=3 dest=Z ttl=1
B -> C: CHAT msgid=3 dest=Z ttl=0
note right of C: TTL = 0\nDROP_TTL

note over A, C: Штатное завершение
A -> B: закрытие TCP-соединения
B -> C: закрытие TCP-соединения
@enduml
```

Топология цепочки из трёх узлов в Graphviz:

```
digraph chain3 {
    rankdir=LR;
    graph [fontname="DejaVu Sans", bgcolor="white"];
    node [shape=box, style="rounded,filled", fillcolor="#e8f0fe",
          fontname="DejaVu Sans", color="#2f5597"];
    edge [fontname="DejaVu Sans", color="#2f5597"];

    A [label="A\n127.0.0.1:20028"];
    B [label="B\n127.0.0.1:20029"];
    C [label="C\n127.0.0.1:20030"];

    A -> B [label="TCP"];
    B -> C [label="TCP"];
}
```

Приложение Г. Сборка и запуск

```
cmake -S lab5 -B lab5/build -DCMAKE_BUILD_TYPE=Release
cmake --build lab5/build -j
ctest --test-dir lab5/build --output-on-failure

./lab5/build/peer_chat --id C --port 20030 --run-ms 60000
./lab5/build/peer_chat --id B --port 20029 --peer 127.0.0.1:20030 \
  --run-ms 60000
./lab5/build/peer_chat --id A --port 20028 --peer 127.0.0.1:20029 \
  --run-ms 60000
```

В интерактивном режиме узла A команда `msg C привет` отправляет сообщение узлу C через промежуточный узел B.

Приложение Д. Полные промпты для раздела об ИИ

Полный текст исходного запроса пользователя сохранён в файле `report/ai_prompts.md`. Он содержит описание лабораторной работы, индивидуальное задание варианта 6, требования к отчёту, экспериментам, Git, оформлению диаграмм и разделу об использовании искусственного интеллекта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Таненбаум Э., Бос Х. Современные операционные системы. — 4-е изд. — СПб.: Питер, 2015.
2. Столлингс У. Операционные системы. — 6-е изд. — М.: Вильямс, 2013.
3. Stevens W. R., Fenner B., Rudoff A. M. UNIX Network Programming. Volume 1: The Sockets Networking API. — 3rd ed. — Addison-Wesley, 2004.
4. POSIX.1-2017. `socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv`, `select`, `shutdown`, `close`, `getaddrinfo`, `setsockopt`.
5. cppreference.com. `std::thread`, `std::mutex`, `std::atomic`, `std::chrono`.
6. Задание на лабораторную работу №5 «Элементы сетевого программирования», вариант 6. БГУИР, 2026.